Feasibility Report: RoutinesBot: AI Chat Assistant for Construction Site

Title: RoutinesBot: AI Chat Assistant for Construction Site

Prepared by: Harshith Muttana,Kirana Subrav,Nithin Nayak

Reviewed by: Prakhyath Rai,Suraj Nagraj

Version: v1.0

1. Executive Summary

The client requires an **AI-powered assistant (chatbot)** to provide fast, contextual, and secure access to its large repository of compliance, procedural, and operational documents, referred to as "Routines". The key challenge is enforcing **fine-grained access control** (Company ID, Project ID, Persona access) during the AI search process while maintaining high performance.

This study proposes implementing a **Retrieval-Augmented Generation (RAG)** solution leveraging **AWS Bedrock** for the Large Language Model (LLM) and a managed **Amazon OpenSearch Service** vector database. This approach is technically feasible and highly suitable as OpenSearch and Bedrock's **Knowledge Bases** feature natively support the required secure, metadata-filtered retrieval.

2. Problem Statement

The client, operating in the construction domain, maintains critical **"Routines"** documents that are mandatory for compliance and internal governance.

The core problems being addressed are:

1. **Information Siloing and Accessibility:** Routines are stored as HTML documents in **Azure Blob Storage**, making them difficult for end-users to quickly find, interact with, and retrieve specific details through traditional search methods.
2. **Contextual Access Requirement:** The routines have a hierarchical structure (Routine → Subroutine) and belong to a **Company, Project, or Global** context, requiring that user queries are **contextually restricted**. Users must only access the routines they are authorized to view based on their Project ID, Company ID, and Persona.
3. **Manual Ingestion:** The process of making new or updated routines available for search is not currently automated, leading to delays and potential non-compliance.
4. **Cross-Cloud Integration:** The new AI system in AWS must securely and automatically integrate with the existing data source in Azure to ingest routine content and metadata.

3. Overview and Objectives

a. Overview

The client maintains regulatory, procedural, and operational documents (Routines) that have multiple versions and contexts (Company, Project, Global). The solution must be an API-driven AI Assistant that allows users to query these

routines via a chat interface, providing relevant information, routine titles, metadata, and a direct link to the source document in Azure.

b. Objectives

- **Provide a Chat-Based AI Assistant** for users to query company/project-specific routines.
- **Automate ingestion** of new or updated routines into an AI knowledge base (Vector DB).
- **Ensure contextual restrictions** – users can only access routines they are authorized to view.
- **Enable linking** between AI search results and the actual Azure-stored routine files.
- **Support multilingual prompts** (Norwegian and English initially).

4. Theory

The solution relies on two key AWS services to implement a sophisticated **Retrieval-Augmented Generation (RAG)** pattern:

AWS Bedrock Service

Amazon Bedrock is a fully managed, serverless service that provides access to a range of high-performing **Foundation Models (FMs)** from leading AI companies (like Anthropic, Meta, and Amazon's Titan family) through a single API.

- **Function:** It serves as the primary generative engine and orchestration layer for the AI Assistant. It hosts the LLM (e.g., Anthropic Claude, Llama 3) for generating human-readable responses and the embedding model (e.g., Titan Text Embeddings V2) for converting text into vectors.
- **RAG Capabilities:** Bedrock's **Knowledge Bases** feature provides a fully managed, end-to-end RAG workflow, automatically handling the splitting of documents (**chunking**), generation of vector embeddings, and storing them in a vector database. It securely connects the FM to the client's proprietary data.

Amazon OpenSearch Service (Vector Database)

Amazon OpenSearch Service is a managed service for deploying and scaling OpenSearch clusters, used for search and analytics.

- **Vector Database Capability:** OpenSearch is utilized as the **vector database** using its k-nearest neighbor (k-NN) vector search

capabilities. It is designed to store the high-dimensional vector embeddings generated by Bedrock.

- **RAG Role:** OpenSearch holds the vector embeddings of the 'Routines' content, indexed alongside the original routine **metadata** (e.g., Company ID, Project ID, Persona access). During a user query, Bedrock sends the user's vector and the required metadata filters to OpenSearch, which quickly retrieves the semantically most similar and *authorized* document chunks. This retrieved context is then passed to the LLM to generate a **factually grounded and accurate** answer, minimizing "hallucinations".

5. Architecture & Cloud Components

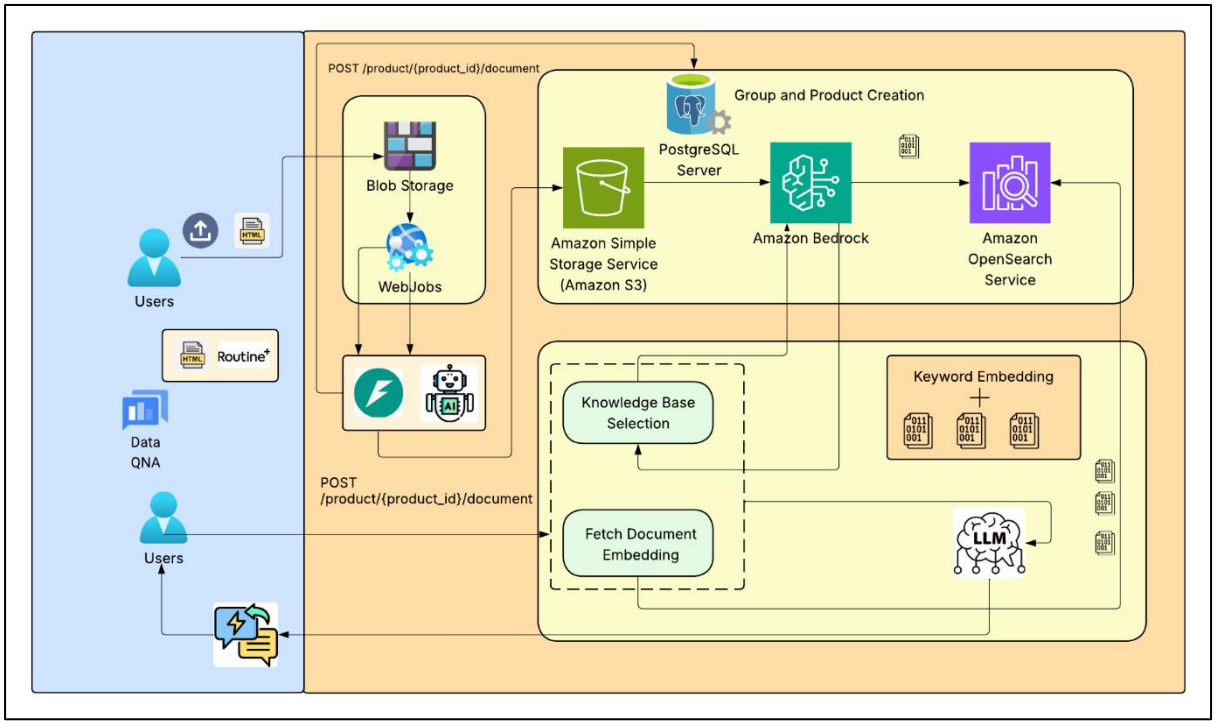

Figure 5 a: High level architecture

The proposed architecture is a **Retrieval-Augmented Generation (RAG)** pipeline utilizing Bedrock Knowledge Bases with an OpenSearch Serverless vector store. This leverages a **cross-cloud integration** to connect the client's data in Azure to the AI services in AWS. The figure 5a as shown in the figure above shows the high-level architecture.

Architecture (RAG Data Flow)

1. **Ingestion Workflow (Azure to AWS):**
    - A new or updated routine is saved in **Azure Blob Storage**.

- For the historical data load, a batch job will be required to migrate data from Azure Blob Storage to S3 and populate the tables. This is a one-time activity that involves migrating historical data from Azure to S3.
- The client's backend application (in Azure) **triggers** an AI ingestion/upload API in AWS.
- **The upload/ingestion API** receives the request and stores the routine's HTML content in a **Amazon S3** bucket.
- **Amazon Bedrock Knowledge Base** initiates a sync/ingestion job, fetching the file from S3, performing **chunking** and **embedding** generation (e.g., using Titan Embeddings).
- The vector embeddings, along with essential **metadata** (Routine ID, Company ID, Project ID, Persona access), are stored in **Amazon OpenSearch Serverless**.

2. **Retrieval Workflow (User Query):**
    - The user submits a query through the chat interface.
    - The request, including the user's session context and metadata (Company ID, Project ID, User Role/Persona), is sent to the **Bedrock Knowledge Base** via the RetrieveAndGenerate API.
    - Bedrock converts the query into an embedding and queries the **OpenSearch Vector Store**, applying the **metadata filters** to ensure contextual security.
    - OpenSearch returns the most relevant, authorized document chunks.
    - Bedrock passes these chunks to the chosen **Foundation Model (FM)** (e.g., Claude 3) to generate a final, grounded answer, including citations and a link to the original routine.

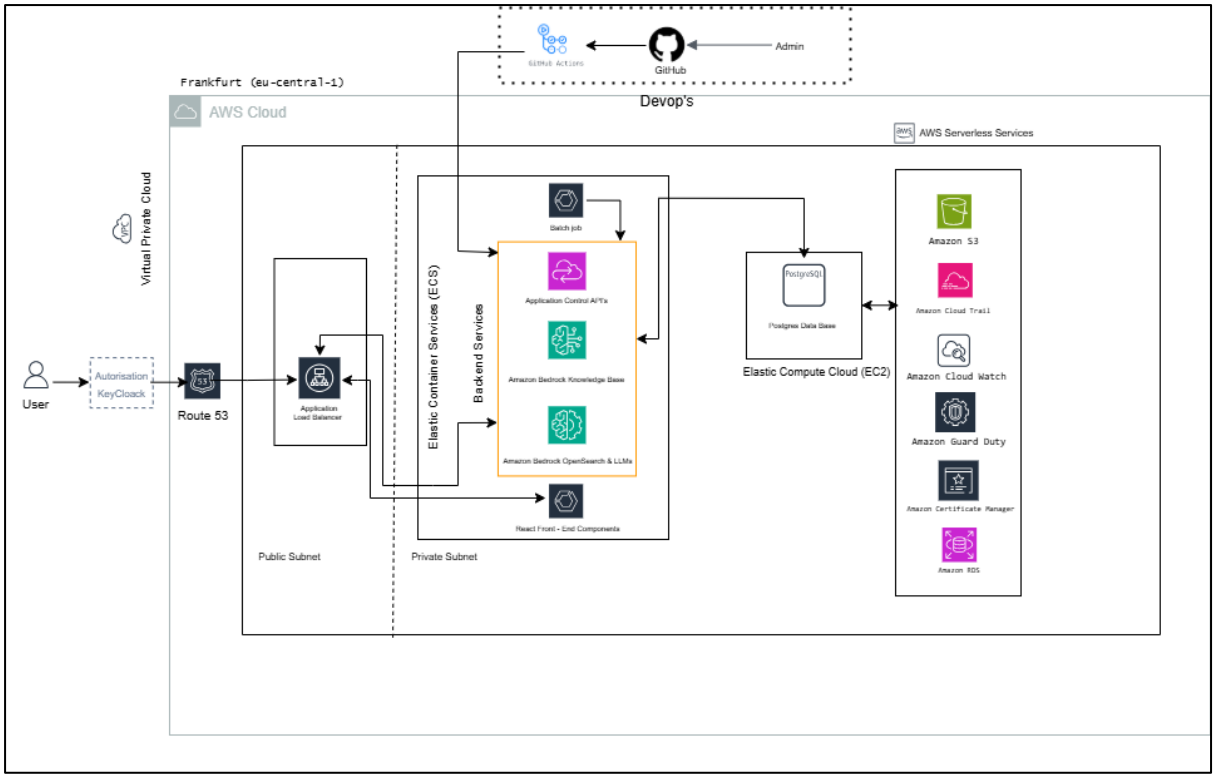

Figure 5 b: EG AI Assistant architecture

The figure 5b shows the architecture for the assistant.

Cloud Components

Component	Function	Justification
Azure Blob Storage	Source of truth for all 'Routines' documents (HTML files).	**Client-side data residency**.
Amazon S3	Staging bucket for Bedrock Knowledge Base ingestion.	**Source data integration** for Bedrock.
Amazon Bedrock	Hosts the **Foundation Models (LLM and Embedding Model)** and the **Knowledge Base** feature.	**Managed GenAI platform** with built-in RAG capabilities.
Amazon OpenSearch Serverless	The **Vector Database** for storing and retrieving high-dimensional embeddings and their metadata.	**Scalable, managed vector store** that natively integrates with Bedrock and supports metadata filtering.

6. Proposed Solution

This will be a **background job** that is **triggered automatically** under the following conditions:

- **Migration** of existing routines from **Azure Blob** to **S3**
- **Creation** of a new routine

- **Update** of an existing routine

The job ensures proper interaction with the **EG AI Assistant APIs**, and updates internal data accordingly to keep the system in sync.

Authentication

Note: client_id, client_secret and tenant_id will be provided from AI COE Team

Authentication

All API calls require a **Bearer token** for authentication.

Obtain Bearer Token:

- Call POST https://api.aicoe.dev.egaiassistant.com/oauth/token with form data:
  - grant_type: client_credentials
  - client_id: Your client ID
  - client_secret: Your client secret
  - audience: Your audience value
- Store the returned access_token for use in subsequent API calls
- Note the expires_in value to refresh the token before expiration

Use Bearer Token:

- Include the token in the Authorization header for all API requests:
  - Format: Authorization: Bearer {access_token}

Note: All API calls in the following steps require this Bearer token in the Authorization header.

Database Schema Updates

Company Table

The following columns are added to track EG AI integration:

- ai_product_id (INTEGER): Product ID from EG AI system
- ai_product_status (VARCHAR(50)): Processing status of the product (e.g., "PROCESSED")
- ai_user_id (TEXT): User ID from EG AI system

DocumentFragmentDbo Table

The following columns are added to track document processing:

- ai_doc_id (TEXT): Document ID from EG AI system
- ai_document_status (VARCHAR(50)): Processing status of the document (e.g., "PROCESSED")

Step-by-Step Execution Logic

Step 1: Create Product and Enable Features

Objective: Create a product for each company in the EG AI system.

Process:

1. Query companies from the database that have documents in documentfragmentdbo table
2. For each company:
    a. Call POST /tenant/{tenant_id}/product with parameters:
        i. product_name: Company ID
        ii. language: The Language of Documents you want to upload in this product(default set en-US)
    b. Store the returned product_id in the company table (ai_product_id column)
    c. Enable citation feature by calling PUT /product/{product_id}/settings with:

```
{
  "features": [
    {
      "feature_name": "CITATION",
      "is_enabled": true
    }
  ]
}
```

Figure 6a: Citation Feature

Error Handling:

- Log failures with company ID and API response details
- Continue processing remaining companies if one fails

Step 2: Create User for Each Product

Objective: Create a widget user associated with each product.

Process:

1. For each successfully created product:
    a. Call POST /tenant/{tenant_id}/user with form data:
        i. user_email: {product_id}@eg.dk
        ii. role: USER
        iii. valid_to: (empty string)
        iv. product_ids: The product ID
    b. Store the returned user_id in the company table (ai_user_id column)
    c. Retrieve the user's API key by calling GET /user/{user_id}/api_key

Error Handling:

- Log user creation failures with product ID and error details
- Retry with exponential backoff for transient failures

Step 3: Validate Product Status

Objective: Ensure the product knowledge base is fully initialized before processing documents.

Process:

1. For each created product:
    a. Poll GET /tenant/{tenant_id}/product/{product_id} endpoint
    b. Check if kb_id (knowledge base ID) is not null in the response
    c. Once kb_id is available:
        i. Update ai_product_status to "PROCESSED" in the company table
        ii. Add product to the processing list
    d. If kb_id is null, wait 20 seconds and retry(polling)

Error Handling:

- Log polling failures with product ID and response details
- Continue polling until successful or maximum retry limit reached

Step 4: Process Routine Fragments (Document Creation)

Objective: Upload routine fragment content as documents to the EG AI ASSISTANT.

Process:

1. Query documents from documentfragmentdbo table where ai_doc_id is NULL
2. Join with company table to get ai_product_id
3. For each routine fragment:
    a. **Skip** if approvedfragmentblobid is null
    b. Retrieve blob content from Azure Blob Storage using:
        i. Container name from container column
        ii. Blob name from approvedfragmentblobid
    c. Get blob properties including content_type
    d. Convert blob data to text format:
        i. Attempt UTF-8 decoding
        ii. Fallback to latin-1 encoding if UTF-8 fails
        iii. Skip document if both encodings fail
    e. Prepare multipart form data with:
        i. language: The Language of the Uploaded Document (Supports en-US(English),da-DK(Danish),nb-NO(Norwegian),sv-SE(Swedish),fi-FI(Finnish),pl-PL(Polish))
        - files: Text content with .txt extension
    - Call POST /product/{product_id}/document with the prepared form data
    - Store the returned document_id in the documentfragmentdbo table (ai_doc_id column)

Error Handling:

- Log blob retrieval failures with blob ID and container name
- Log document upload failures with fragment ID and API response
- Skip failed documents and continue processing
- Retry transient failures with exponential backoff

Step 5: Validate Document Processing Status

Objective: Ensure documents are fully processed and indexed in the knowledge base.

Process:

1. For each uploaded document:
    a. Poll GET /product/{product_id}/documents endpoint with parameters:
        i. source_type: FILE
        ii. limit: 10
        iii. offset: 0
    b. Filter the response to find the document matching the uploaded document_id
    c. Check if processed_date is not null:
        i. If null: wait 10 seconds and retry
        ii. If not null:
            1. Update ai_document_status to "PROCESSED" in documentfragmentdbo table
            2. Add document to the processing list and proceed to next document

Error Handling:

- Log polling failures with document ID and response details
- Wait 20 seconds before retrying if API call fails

Continue polling until document is processed

Step 6: Generate Widget Chat Session

Objective: Create an interactive chat widget for users to query their documents at company or project level.

6.1 Project-Level Chat Widget

When to use: When user wants to chat with documents from specific project(s) only.

Process:

1. Join documentinfodbo, documentfragmentdbo, and company tables to get:
    a. Project IDs and their associated document IDs
    b. Product ID and User ID from company table
2. Filter data by:
    a. companyid: The specific company UUID
    b. projectid: One or more project UUIDs (provided as list)
3. Collect all ai_doc_id values for the filtered projects
4. **Optional: Configure Guardrails**
    a. Retrieve default agent by calling GET /product/{product_id}/ai/default-agent
    b. Update guardrail settings by calling PUT /agents/{agent_id}/guardrail with configuration for:
        i. Content filters (SEXUAL, VIOLENCE, HATE, INSULTS, MISCONDUCT, PROMPT_ATTACK)
        ii. PII entity protection (credit cards, emails, phone numbers)
        iii. Grounding and relevance checks
5. Call POST /widget/generate with as shown in the figure 6.1 a:

```
{
  "doc_ids": ["list of document IDs from selected projects"],
  "role": "WIDGET_USER",
  "user_id": "user ID from company table",
  "expire": 1800,
  "title": "ConstructionSite",
  "language_culture_code": "en-US",
  "external_user_identifier": "user email"
}
```

Figure 6.1 a: Widget Payload

2. Generate and return the widget URL:
   https://aicoe.dev.egaiassistant.com/widget?t={token}&uid={user_id}

Example Usage:

```
create_ai_chat_for_project_or_company(
    companyid='44f6afc7-f2c6-4f6a-9081-8ddbf1ca2f4c',
    projectids=['8fcd69bf-cd45-4d9c-ba0b-8caf0756fd99', '00000000-0000-0000-0000-000000000000']
)
```

Figure 6.1 b: Example Usage Of Chat Function

6.2 Company-Level Chat Widget

When to use: When user wants to chat with all documents across the entire company.

Process:

1. Join documentinfodbo, documentfragmentdbo, and company tables
2. Filter data by companyid only (no project filtering)
3. Get unique ai_product_id and ai_user_id for the company
4. **Optional: Configure Guardrails** (same as project-level)
5. Call POST /widget/generate with:

```
{
  "product_id": "product ID from company table",
  "role": "WIDGET_USER",
  "user_id": "user ID from company table",
  "expire": 1800,
  "title": "ConstructionSite",
  "language_culture_code": "en-US",
  "external_user_identifier": "user email"
}
```

Figure 6.2 a: Company Level Chat Widget

Note: When using product_id, all documents in the product's knowledge base are accessible

6. Generate and return the widget URL: https://aicoe.dev.egaiassistant.com/widget?t={token}&uid={user_id}

Example Usage:

```
create_ai_chat_for_project_or_company(
    companyid='44f6afc7-f2c6-4f6a-9081-8ddbf1ca2f4c'
)
```

Figure 6.2 b: Example Usage of Chat Function

6.3 Key Differences

Aspect	Project-Level Chat	Company-Level Chat
Scope	Specific project(s) only	All company documents
API Parameter	doc_ids (array of document IDs)	product_id (single product ID)
Document Filtering	Explicit list of documents	All documents in knowledge base
Use Case	Targeted collaboration on specific projects	Organization-wide knowledge access

6.4 Optional Guardrail Configuration

Purpose: Add safety controls and content filtering to AI responses.

APIs Used:

1. GET /product/{product_id}/ai/default-agent - Retrieve the default agent ID
2. PUT /agents/{agent_id}/guardrail - Update guardrail configuration

Configuration Options:

- **Grounding**: Ensure responses are based on provided documents
- **Relevance**: Filter out off-topic responses
- **Content Filters**: Block harmful content (sexual, violent, hateful, insulting, misconduct)
- **Prompt Attack Protection**: Defend against prompt injection attempts
- **PII Protection**: Block credit card numbers, emails, phone numbers
- **Custom Messages**: Set blocked input/output messages

When to Use:

- Enable guardrails when safety and compliance are critical
- Configure based on organizational policies and use case requirements
- Can be skipped for internal testing or trusted environments

Error Handling:

- Log guardrail update failures but continue with widget generation
- Guardrail configuration is independent of chat functionality

Validation:

- Validate all document IDs are non-null before generating widget
- Verify company ID and product ID exist before API calls
- Exit gracefully if no valid product ID or user ID found

Error Logging:

- Log widget generation failures with user ID, company ID, project IDs (if applicable), and error details
- Include query results in logs for debugging

Data Flow Summary

1. **Company** → Creates → **Product** (with Citations enabled) → Creates → **User**
2. **Product** → Validates → **Knowledge Base** (kb_id available) → Status: PROCESSED
3. **Document Fragments** → Upload → **Documents** → Validates → **Processing** → Status: PROCESSED
4. **Documents** → Filter by → **Company/Project** → Optional → **Guardrails** → Generate → **Widget Chat Session**

Error Handling and Retries

General Principles:

1. **Logging:** All API failures are logged with:
    a. Entity ID (company, fragment, project, etc.)
    b. API endpoint called
    c. HTTP status code
    d. Response body

    e. Relevant query results
2. **Retries:** Transient failures (network issues, 5xx errors) are retried with:
    a. Fixed delay strategy (10-20 seconds between polls)
    b. Exponential backoff for API call failures
    c. Maximum retry attempts based on operation type
3. **Graceful Degradation:** If a required field is missing or an entity fails:
    a. Skip that specific item
    b. Log the issue with context
    c. Continue processing remaining items
    d. Exit only when critical prerequisites are missing
4. **Validation:** Before API calls:
    a. Verify required fields are non-null (ai_product_id, ai_doc_id, approvedfragmentblobid)
    b. Validate data format and UUID validity
    c. Check data type conversions (string casting for IDs)

Output and Status Tracking

After successful execution:

- **Company records** updated with:
    - ai_product_id: Product identifier in EG AI
    - ai_product_status: "PROCESSED" when knowledge base is ready
    - ai_user_id: Widget user identifier
- **Document fragments** updated with:
    - ai_doc_id: Document identifier in EG AI
    - ai_document_status: "PROCESSED" when indexing is complete
- **Processing logs** contain:
    - Success/failure status of each operation
    - Product and document processing lists
    - Generated widget URLs with tokens
- **Widget URLs** generated for:
    - Project-specific document access
    - Company-wide document access
    - Time-limited sessions (1800 seconds = 30 minutes)(can be extended)

7. Risk Analysis

The proposed architecture relies on the seamless interaction of multiple cloud services and RAG mechanisms. Key risks associated with this solution include:

Risk Area	Description	Impact	Mitigation Strategy
LLM Hallucination/Inaccuracy	The Large Language Model (LLM) may generate answers that are factually incorrect but convincing (hallucinations), critical for regulatory/compliance docs.	**High.** Risks non-compliance or incorrect procedures being followed by users.	**Grounded Responses:** Enforce strict prompt engineering to force the LLM to only use retrieved, cited context. Always return direct routine link and source citations.
Multilingual Performance	The selected Foundation Model (FM) may struggle with complex queries or nuances in Norwegian, impacting "Multilingual Support".	**Medium.** Reduces user satisfaction and utility for a segment of users.	**Model Evaluation:** Conduct focused language performance evaluation during POC to select the best-performing FM for both English and Norwegian.

8. Limitations

The following limitations are inherent to the current design or the underlying RAG technology:

- **Reliance on Metadata Integrity:** The entire security model and contextual access hinges on the accuracy and completeness of the metadata (**Company ID, Project ID, Persona Access, Language**) stored alongside the routine content . Any corruption or omission in this metadata will directly compromise the contextual access rule.
- **Cost Predictability:** Both Amazon Bedrock (per token) and OpenSearch Serverless (per usage unit) are based on consumption. Under unexpectedly high

user query load or frequent, large ingestion jobs, the operational cost could exceed budget projections.

- **LLM Reasoning:** While the RAG system excels at *retrieval* and *summarization*, the LLM's ability to fulfill complex functional requirements like "Suggest which routines to use or import" based on nuanced, comparative policy logic remains a constraint and requires careful prompt design.

9. Conclusion

The feasibility study concludes that building the AI-Powered Routines Assistant using **AWS Bedrock Knowledge Bases** integrated with **Amazon OpenSearch Serverless** as the vector store is **highly feasible** and the **recommended architectural approach**.

The architecture directly addresses all key functional and non-functional requirements:

- **Contextual Security:** The use of OpenSearch's vector search with strict **metadata filtering** ensures users are restricted to routines based on their Company ID, Project ID, and Persona.
- **Automation:** The API-driven ingestion flow allows for the "Automatic ingestion" of new or updated routines directly from the Azure backend.

The primary risk is the **cross-cloud integration** and ensuring absolute **security** through metadata filtering. The project should immediately proceed to a **Proof of Concept (POC)** phase focused on rigorously testing these high-risk areas to validate the architecture before full-scale development.